Control statements

03 May 2025 09:54

PDF

Control Statem...

Control Statements

01 May 2025 18:20

Java Compiler executes the code from top to bottom. The statements in the code are executed according to the order in which they appear. However, Java provides statements that can be used to control the flow of Java code. Such statements are called control flow statements. It is one of the fundamental features of Java, which provides a smooth flow of program.

Fundamentally, Java provides three types of control flow statements:

1) Decision Making Statements
   a) if statement
   b) switch statement
2) Looping statement
   a) While loop
   b) do...while loop
   c) for loop
   d) for-each loop
3) Jump statements
   a) break
   b) continue

1. Decision making statements: As the name suggests, decision making statements decide which statement to execute and when. decision making statements evaluate the Boolean expression and control the program flow depending upon the result of the condition provided. There are two types of decision making statements in Java. That is, if statement and switch statement.

   "if statement": In Java, the if statement is used to evaluate a condition. the control of the program is diverted depending upon the specific condition. The condition of the if statement gives a boolean value, either true or false. In Java, there are four types of if statement given below:

   i. Simple if statement
   ii. if-else statement
   iii. if-else-if statement
   iv. Nested if statement.

i. **Simple if statement:**

It is the most basic statement among all control flow statement in Java. It evaluates a Boolean expression and enables the program to enter a block of code if the expression evaluates to true.

Syntax:

```
if (<condition>){
    Statement1; //executes when condition is true
}
```

For example:

```
public class One
{
  public static void main(String args[]){
    int x = 10;
    int y = 10;
    if(x+y > 20){
      System.out.print("\fx+y greater than 20");
    }
  }
}
```

New Section 1 Page 1

ii. **if-else statement:** The if else statement is an extension of the if statement which uses another block of code that is else block. the else block is executed if the condition of the if block is evaluated as false.

Syntax:

```
if (<condition>){
    Statement1; //executes when condition is true.
}
else{
    Statement2; //executes when condition is false.
}
```

For example:

```
public class One
{
  public static void main(String args[]){
    int x = 10;
    int y = 10;
    if(x+y > 20){
      System.out.print("\fx+y greater than 20");
    }
    else{
      System.out.print("\fx+y not greater than 20");
    }
  }
}
```

Output:



iii. **if-else-if ladder:** The if-else-if statement contains the if-statement followed by multiple else-if statements. In other words, we can say that it is a chain of if-else statement that create a decision tree where the program may enter in the block of code where the condition is true. We can also define an else statement at the end of the chain.

Syntax:

```
if(<condition 1>){
    Statement 1; //executes when condition 1 is true.
}
else if(condition 2){
    Statement 2; //executes when condition 2 is true.
}
else if(condition 3){
    Statement 3; //executes when condition 3 is true.
}
...
else if(condition N){
    Statement N; //executes when condition N is true.
}
else{
    Statement; //executes when all the conditions are false.
```

```
    }
```

For example:

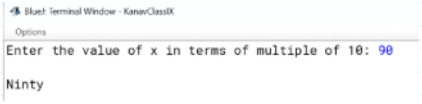
```
package ControlStructure.IfElseStatement;
import java.util.*;


/**
 * Write a description of class Two here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Two
{
   public static void main(String args[]){
      Scanner sc = new Scanner(System.in);
      int x;
      System.out.print("\fEnter the value of x in terms of multiple of 10: ");
      x = sc.nextInt();
      if( x == 10){
         System.out.print("\nTen");
      }
      else if(x == 20){
         System.out.print("\nTwenty");
      }
      else if(x == 30){
         System.out.print("\nThirty");
      }
      else if(x == 40){
         System.out.print("\nForty");
      }
      else if(x == 50){
         System.out.print("\nFifty");
      }
      else if(x == 60){
         System.out.print("\nSixty");
      }
      else if(x == 70){
         System.out.print("\nSeventy");
      }
      else if(x == 80){
         System.out.print("\nEighty");
      }
      else if(x == 90){
         System.out.print("\nNinty");
      }
      else{
         System.out.print("\nNot a multiple of 10.");
      }

   }
```

```
        }
```

Output:



iv. **Nested if-statement:** The nested if statement. if a statement contains a if or if else statement inside another if or else if statement.

Syntax:

```
if(condition 1){
    Statement 1; //executes when condition 1 is true
    if(condition 2){
        Statement 2; //executes when condition 2 is true
    }
    else{
        Statement 3; //executes when condition 2 is false
    }
}
else{
    Statement 4; //executes when condition 1 is false
    if(condition 3){
        Statement 5; //executes when condition 3 is true
    }
    else{
        Statement 6; //executes when condition 3 is false
    }

}
```

For example:

```
package ControlStructure.IfElseStatement;
import java.util.*;


/**
 * Write a description of class Three here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Three
{
   public static void main(String args[]){
      Scanner sc = new Scanner(System.in);
      int var;
      System.out.print("\fEnter any number: ");
      var = sc.nextInt();
      if(var >= 0){
         if(var % 2 == 0){
```

```
            System.out.print("\nEven number");
          }
          else{
            System.out.print("\nOdd number");
          }
        }
        else{
          System.out.print("\nNegative number is entered.");
        }
      }
    }
```

Output:

1.
```
Blue: Terminal Window - KanavClassIX
Options
Enter any number: 45

Odd number
```

2.
```
Blue: Terminal Window - KanavClassIX
Options
Enter any number: -10

Negative number is entered.
```

3.
```
Blue: Terminal Window - KanavClassIX
Options
Enter any number: 10

Even number
```


1. Write a program to input three angles of a triangle and check whether its construction is possible or not. If possible then check and display whether it is an acute-angled triangle, right-angled or an obtuse-angled triangle. Otherwise, display 'A Triangle is not possible'.
Sample Input: Enter three angles: 40, 50, 90
Sample Output: Right–Angled Triangle

Inputs → A, B, C

$A + B + C = 180^\circ$

$180^\circ + 0^\circ + 0^\circ = 180$ → St. line

construction is not possible if any of the angle value is 0

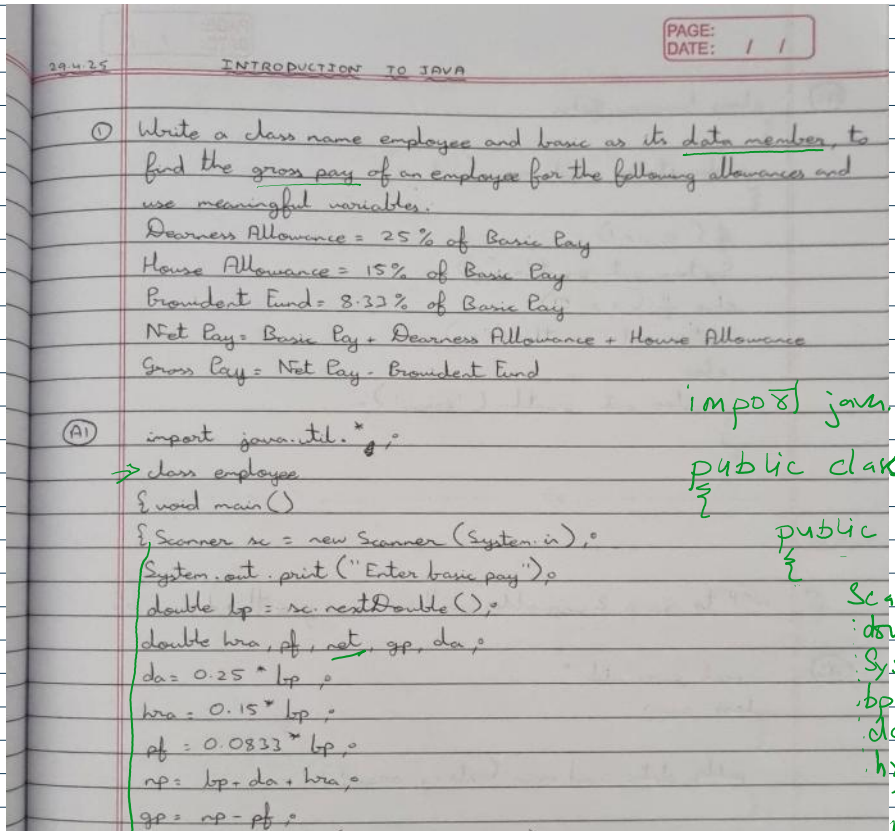
or

sum of 3 angles $!= 180^\circ$

```
package ConditionalStatements;
import java.util.Scanner;
```

```java
/**
 * Write a description of class Q1 here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Q1
{
    public static void main(String args[]){
        Scanner sc = new Scanner(System.in);
        int a, b, c, sum;
        System.out.print("\fEnter first angle: ");
        a = sc.nextInt();
        System.out.print("Enter second angle: ");
        b = sc.nextInt();
        System.out.print("Enter third angle: ");
        c = sc.nextInt();

        sum = a + b + c;
        if(a == 0 || b == 0 || c == 0 || sum != 180){
            System.out.print("\nConstruction of trianle is not possible.");
        }
        else{
            //construction of triangle is possible
            if(a < 90 && b < 90 && c < 90){
                System.out.print("\nAcute-angle triangle");
            }
            else if( a > 90 || b > 90 || c > 90){
                System.out.print("\nObtuse-angle triangle");
            }
            else{
                System.out.print("\nRight-angle triangle");
            }
        }
    }
}
```



29.4.25 INTRODUCTION TO JAVA

PAGE: DATE: / /

① Write a class name employee and basic as its data member, to find the gross pay of an employee for the following allowances and use meaningful variables.
Dearness Allowance = 25% of Basic Pay
House Allowance = 15% of Basic Pay
Provident Fund = 8.33% of Basic Pay
Net Pay = Basic Pay + Dearness Allowance + House Allowance
Gross Pay = Net Pay - Provident Fund

(A1)
```
import java.util.*;
class employee
{void main()
{Scanner sc = new Scanner (System.in);
System.out.print("Enter basic pay");
double bp = sc.nextDouble();
double hra, pf, net, gp, da;
da = 0.25 * bp;
hra = 0.15 * bp;
pf = 0.0833 * bp;
np = bp + da + hra;
gp = np - pf;
```

[ ] ← bracket
( ) ← parenthesis
{ } ← curly brace

```
import java.util.*;
public class Employee
{
public static void main (String args[])
{
Scanner sc = new Scanner(System.in);
double bp, hra, pf, net, gp, da;
System.out.print("Enter basic pay:");
bp = sc.nextDouble();
da = 0.25 * bp;
hra = 0.15 * bp;
pf = 0.0833 * bp;
np = bp + da + hra;
```

```
pf = 0.0833 * bp;
np = bp + da + hra;
gp = np - pf;
System.out.print("Gross Pay" = + gp);
}}
```

② Input a binary number (0/1) and print the other no.

Example -

Input: 1 Output: 0

Input: 0 Output: 1

```
hra = 0.15 * bp;
pf = 0.0833 * bp;
np = bp + da + hra;
gp = np - pf;
System.out.print("\n Gross Pay = " + gp);
}
}
```

PAGE:
DATE: / /

(A2)

```
class binary_ifelse
{
  public static void main (int x)
  {
    if (x==1)
    System.out.println("0");
    else if (x==0)
    System.out.println("1");
    else
    System.out.println("error");
  }
}
```

```
if (x==1)
    System
else if (x==0)
    System
else
    System
```

5.5.25

③ WAP to swap 2 variables without using a third variable.

(A3)

```
import java.util.*
class swap
{
  public static void main (String args[])
  {
    Scanner sc = new Scanner (System.in);
    int a = sc.nextInt();
    int b = sc.nextInt();
    a = a+b;
    b = a-b;
    a = a-b;
    System.out.println("A" + a + "B" + b);
  }
}
```

a 5 b 10

From <https://teams.live.com/v2/>

```
package Chapter0;
import java.io.*;
```

```
/**
 * Write a description of class DemoOfInput here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class DemoOfInput
{
    public static void main(String args[]) throws IOException {
        InputStreamReader isr = new InputStreamReader(System.in);
        System.out.print("\fEnter any number: ");
        int x = isr.read();
        System.out.print("\nx = " + x);
    }
}
```

read() always read first character

'0' = 48
'1' = 49
'2' = 50
'3' = 51
'4' = 52
'5' = 53
'6' = 54.
'9' = 57

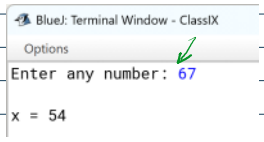
Unicode value of

'A' = 65
'B' = 66
'Z' = 90
'a' = 97
'z' = 122

'a' 'A'
97 - 65

Space bar = '32'

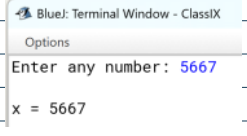
Output:




```
package Chapter0;
import java.io.*;


/**
 * Write a description of class DemoOfInput here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class DemoOfInput
{
    public static void main(String args[]) throws IOException {
        InputStreamReader isr = new InputStreamReader(System.in);
        BufferedReader br = new BufferedReader(isr);
        System.out.print("\fEnter any number: ");
        //int x = isr.read();
        int x = Integer.parseInt(br.readLine());
        System.out.print("\nx = " + x);
    }
}
```

Output:



Java Switch Statement

The Java switch statement executes one statement from multiple conditions. It is like if else ladder statement. The switch statement works with byte, short, int long. enum types string and sum wrapper types like Byte, Short, Integer, Long. Since Java 7 we can use strings in the switch statement.

The switch statement can be described as a control flow type statement, which is utilized for manipulating the flow of program, executing and invoking various branches of code using the value of an expression.

In other words, the switch statement tests the equality of a variable against multiple values.

Points to Remember

- There can be one or n number of case values for a switch expression.
- The case value must be of switch expression type only. The case value must be literal or constant. It does not allow variables.
- The case value must be unique. in case of duplicate value, it renders compile time error.
- Each case statement can have a break statement, which is optional. When control reaches to the break statement, it jumps the control. After the switch expression, if the break statement is not found, it executes the next case. This type of code is called fall through.
- The case value can have a default label, which is optional.

In Java, switch statement mainly provides a more detailed alternative that avoids the users of nested or several if else statements well associated with an individual variable.
The syntax of the Java switch statement contains the switch keyword, which is followed by the expression that needs to be evaluated using parenthesis. The mentioned expression must definitely evaluate to a definite data type, which is primitive, such as int, char, string.

Syntax:

```
switch(expression){
   case value1:{
     //code to be executed
     break;
    }
    case value2:
   {
     //code to be executed
     break;
    }
   …
   case valueN:
   {
     //code to be executed
     break;
    }
   default:
     //Code to be executed
}
```

Example code:

```
package ConditionalStatements;
import java.util.*;

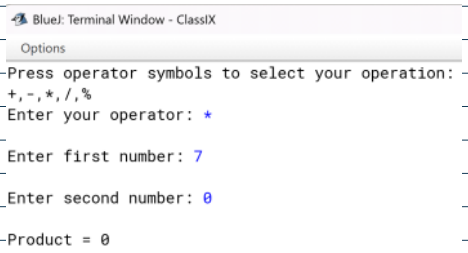

/**
```

```
 * Write a description of class SwitchDemo here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class SimpleCalculator{
    public static void main(String args[]){
        Scanner sc = new Scanner(System.in);
        int x, y, r;
        char oper;
        System.out.print("\fPress operator symbols to select your operation: ");
        System.out.print("\n+,-,*,/,%");
        System.out.print("\nEnter your operator: ");
        oper = sc.next().charAt(0);

        switch(oper){
            case '+':
                System.out.print("\nEnter first number: ");
                x = sc.nextInt();
                System.out.print("\nEnter second number: ");
                y = sc.nextInt();
                r = x + y;
                System.out.print("\nSum = " + r);
                break;
            case '-':
                System.out.print("\nEnter first number: ");
                x = sc.nextInt();
                System.out.print("\nEnter second number: ");
                y = sc.nextInt();
                r = x - y;
                System.out.print("\nDifference = " + r);
                break;
            case '*':
                System.out.print("\nEnter first number: ");
                x = sc.nextInt();
                System.out.print("\nEnter second number: ");
                y = sc.nextInt();
                r = x * y;
                System.out.print("\nProduct = " + r);
                break;
            case '/':
                System.out.print("\nEnter first number: ");
                x = sc.nextInt();
                System.out.print("\nEnter second number: ");
                y = sc.nextInt();
                if(y == 0){
                    System.out.print("\nDivisor can't be zero.");
                    break; //it will break the switch
                }
                r = x / y;
                System.out.print("\nQuotent = " + r);
                break;
            case '%':
                System.out.print("\nEnter first number: ");
                x = sc.nextInt();
                System.out.print("\nEnter second number: ");
                y = sc.nextInt();
                if(y == 0){
                    System.out.print("\nDivisor can't be zero.");
                    break; //it will break the switch
                }
                r = x % y;
                System.out.print("\nRemainder = " + r);
                break;
```

```
            default:
                System.out.print("\nWrong operator selected.");
        }
    }
}
```

Output:

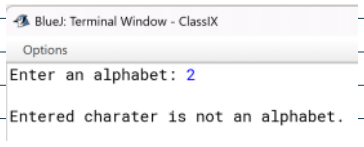


Fall through: The break statement is not applied after every case. Then the execution control will jump to next case automatically, and continues to fall till it gets the break statement.

```
package ConditionalStatements;
import java.util.*;

/**
 * Write a description of class VowelChecker here.
 *
 * @author (Shiv)
 * @version (1.0 14-05-2025)
 */
public class VowelChecker
{
    public static void main(String args[]){
        Scanner sc = new Scanner(System.in);
        System.out.print("\f");
        char c;
        System.out.print("Enter an alphabet: ");
        c = sc.next().charAt(0);
        if((c >= 'A' && c <= 'Z')||(c >= 'a' && c <= 'z')){
            switch(c){
                case 'A': case 'E': case 'I': case 'O': case 'U':
                case 'a': case 'e': case 'i': case 'o': case 'u':
                    System.out.print("\nThe entered character is vowel.");
                    break;
                default:
                    System.out.print("\nEntered character is a consonant.");
            }
        }
        else{
            System.out.print("\nEntered charater is not an alphabet.");
        }
    }
}
```

Outputs:



```
BlueJ: Terminal Window - ClassIX
Options
Enter an alphabet: u

The entered character is vowel.
```

```
BlueJ: Terminal Window - ClassIX
Options
Enter an alphabet: y

Entered character is a consonant.
```

```
10 public class Dice
11 {
12     public static void main (String args[])
13     {
14         Scanner sc=new Scanner(System.in);
15         System.out.print("\f");
16         System.out.println("Random:"+(int)((Math.random()*10)%6+1));
17     }
```

Probable remainder

{0, 1, 2, 3, 4, 5}

{1, 2, 3, 4, 5, 6}

0.8786215

(8.786215 % 6)

2.786215 + 1

3 ~~.786215~~

```
package ConditionalStatements;
import java.util.*;


/**
 * Write a description of class Dice here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Dice
{
    public static void main(String args[]){
        Scanner sc = new Scanner(System.in);
        System.out.print("\f");
        System.out.print("Dice rolled: " + new Random().nextInt(1, 7));
    }
}
```

Home Work: Questions given by the school